SIMILARITY-BASED DUPLICATE RECORD DETECTION SYSTEM

Final Project Review | Department of Computer Science and Engineering | K L (Deemed to be) University

TEAM MEMBERS

2420030540 – ALLIPURAM AKHIL

2420030440 – D. SURYA

2420030546 – P. RUTHVIK REDDY

Guide: Ms. Bonagani Prathusha, Assistant Professor | Academic Year: 2026–27 (Odd Sem)

Introduction & Problem Statement

Background

Customer data arrives from web forms, apps and legacy systems, so one person is often entered twice with typos, reformatted phones or abbreviated addresses.

Problem Statement

Exact-match checks miss near-duplicates and comparing every pair grows quadratically. We need multi-field scoring that ends in a cleaned dataset.

Motivation / Need

Duplicates split customer histories, inflate counts and raise storage cost. Analysts and CRM teams need a traceable, reviewable path to clean data.

One customer, two rows

RECORD A

Karthik Devi
karthik.devi44@gmail.com
9996734137
485, Gachibowli, Bengaluru

Same person? Exact match says no

RECORD B

Karthik Devi
karthik.devi44@gmail.com
999-673-4137
485, Gachibowli, Bengaluru

Normalisation exposes the match

Objectives & Scope

Objectives

1. Normalise records and flag rows with missing fields before comparison
2. Generate candidate pairs by blocking instead of comparing all 126,253 pairs
3. Compute five similarity feature types and compare Logistic Regression, Random Forest and XGBoost
4. Cluster duplicates with union-find; deliver a cleaned dataset and an interactive dashboard

In Scope

- Structured customer records (name, email, phone, address)
- Full pipeline: normalise, block, score, classify, cluster
- Pair report, cleaned CSV, metrics file and dashboard

Out of Scope

- Proof of accuracy on real customer databases (data is synthetic)
- Transformer matching and human-review queue (future work)
- Production security, audit logging and scaling

Literature Survey

S.No	Author(s) & Year	Method / Technique	Key Findings	Limitation / Gap
1	Fellegi & Sunter, 1969	Probabilistic record linkage	Weigh agreement across several fields together	No learned string similarity
2	Cohen et al., 2003	Comparison of string distance metrics	Best metric depends on the data	No single metric is enough
3	Christen, 2012; Papadakis et al., 2020	Blocking and indexing surveys	Shrink the search space of all pairs	Strict blocks can drop true duplicates
4	Bilenko & Mooney, 2003	Learnable string similarity	Pair features can be learned from labels	Stops at pairs; no clusters or cleaned data

Research Gap

Existing work rarely turns pair predictions into entity clusters and a cleaned dataset, or keeps pair counts and row counts separate. Our pipeline closes this gap with an inspectable, end-to-end workflow (objectives O1–O7).

Proposed System & Methodology

STEP 1

Ingest & Clean

Validate CSV; lowercase text, digits-only phones, flag missing.

STEP 2

Block

Key + neighbouring-name pairs: 126,253 → 605 candidates.

STEP 3

Score

Levenshtein, Jaro–Winkler, Jaccard, TF–IDF and exact-match features.

STEP 4

Classify

Logistic Regression, Random Forest and XGBoost on one feature set.

STEP 5

Consolidate

Union-find clusters, one representative each, dashboard and export.

Key Novelty / Contribution

An inspectable end-to-end pipeline from raw CSV to clusters and a cleaned dataset, reporting pair counts and row counts separately and comparing three classifiers on one shared feature set.

System Architecture

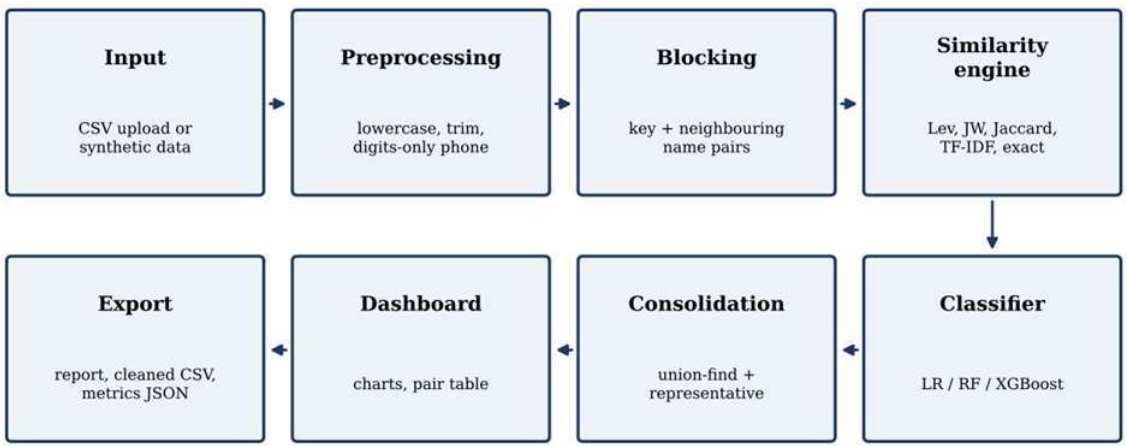


Fig. 4.1: Functional architecture of the duplicate detection pipeline

Pair-level matching is kept separate from record-level cleaning: 172 flagged links removed only 146 rows, because links overlap inside clusters.

Components

1. **Preprocessing**
   Lowercase, trim, digits-only phone
2. **Blocking**
   Key + neighbouring-name pairs
3. **Similarity engine**
   11 pair features incl. combined score
4. **Classify & consolidate**
   LR / RF / XGBoost + union-find

Hardware & Software Requirements

Hardware

- Intel Core i3 / i5 or equivalent
- RAM: 4 GB min, 8 GB recommended
- Storage: 1 GB free space
- GPU not required

Software & Tools

- Windows 10/11 or any OS with Python 3.x
- Python 3.x
- VS Code or Jupyter Notebook
- Python virtual environment

Libraries / Frameworks

- pandas, NumPy
- scikit-learn, XGBoost
- Python-Levenshtein, jellyfish
- Flask, Streamlit, Plotly

Deployment / Platform: Local machine – Flask app and Streamlit dashboard in one Python environment (503-record dataset)

Dataset / Data Understanding

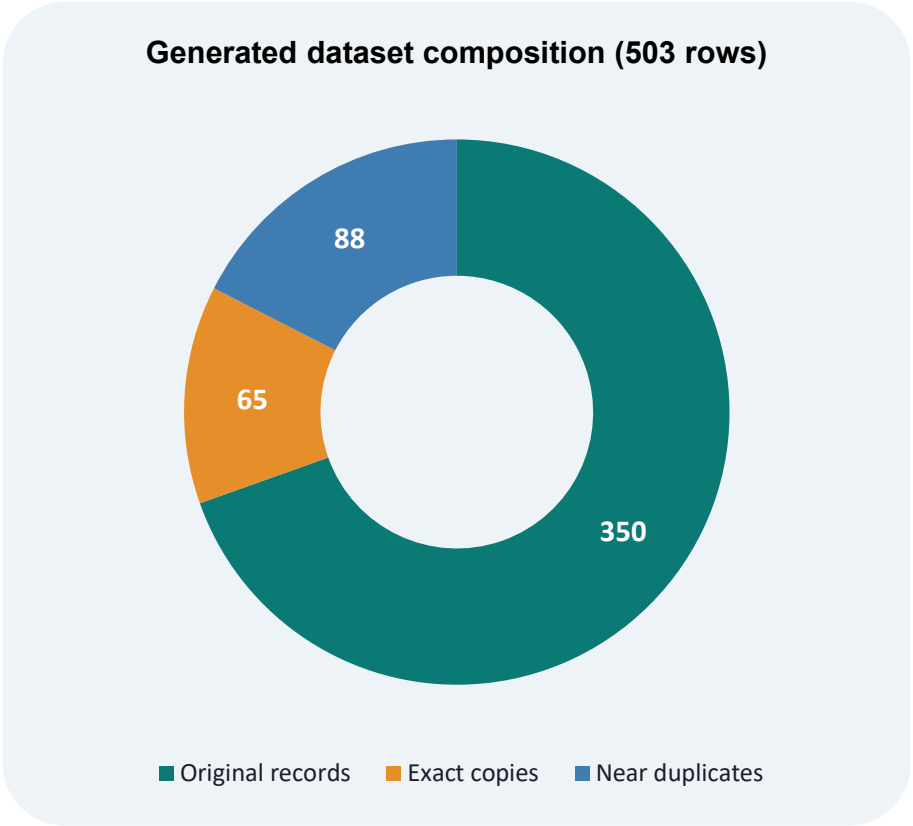
503
Records (rows)

6
Fields (columns)

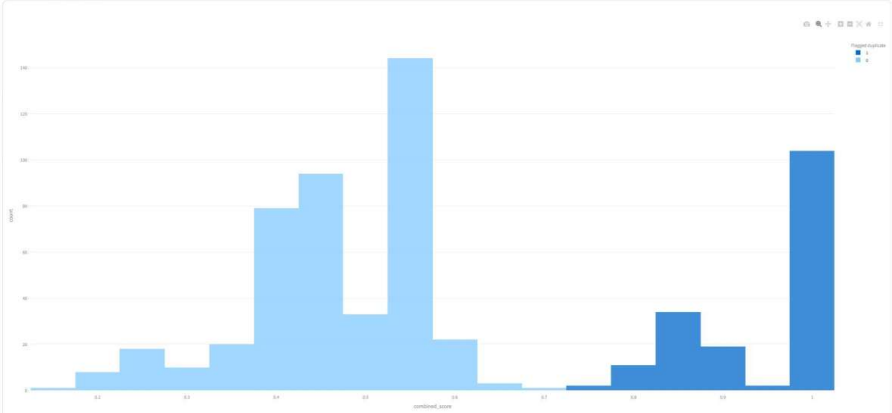
18
Rows with a missing field

2
Pair classes (dup / not)

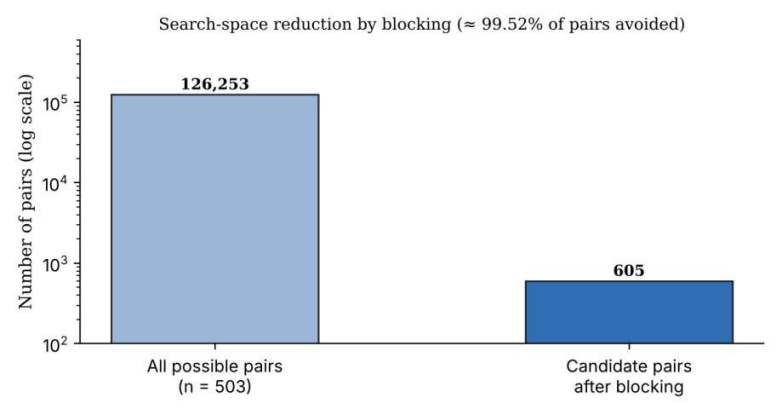
- Source: generated customer data – 350 originals + 153 injected duplicates
- Fields: id, name, email, phone, address, duplicate_of (labels only)
- Quality: 65 exact copies, 88 near duplicates, 122 source records duplicated



Data Preprocessing & Exploratory Analysis



Distribution of combined similarity scores



Blocking cuts all-pairs comparisons by ≈ 99.5%

Cleaning

Missing values become empty strings and the row is flagged; two empty fields never count as a match.

Transformation

Lowercase, collapse spaces, drop symbols; phones keep digits only (999-673-4137 → 9996734137).

Feature Selection

11 pair features; duplicate_of is used only for labels, so there is no target leakage.

Module-wise Description

1 Ingestion & Preprocessing

Purpose: Load, validate and normalise the CSV

Input → Output: Raw records → normalised records + flags

Developed by: All team members

2 Candidate Generation

Purpose: Build candidate pairs by blocking

Input → Output: Normalised records → 605 candidate pairs

Developed by: All team members

3 Similarity & Classification

Purpose: Compute features; train and compare 3 models

Input → Output: Candidate pairs → features, labels, metrics

Developed by: All team members

4 Consolidation & Dashboard

Purpose: Cluster links; show charts, pairs, downloads

Input → Output: Predicted pairs → cleaned dataset + dashboard

Developed by: All team members

Algorithms & Technologies Used

Logistic Regression

Why chosen
Interpretable baseline with a linear decision boundary

Key parameters
Decision threshold on P(y=1 | x)

Random Forest

Why chosen
Captures non-linear interactions, e.g. name similarity plus phone match

Key parameters
Bootstrapped trees, random feature subsets

XGBoost

Why chosen
Efficient boosted trees for tabular data, with regularisation

Key parameters
Sequential trees; sensitive to tuning

Selection criteria: all three use the same features; rounded metrics tie (F1 0.990), so we call them similar and declare no winner

Implementation / Model Building

Feature Selection

11 features: Levenshtein, Jaro–Winkler, Jaccard, TF–IDF cosine, exact phone/email, combined score

Preprocessing Pipeline

Validate IDs, fill missing with empty strings, normalise text, digits-only phones

Data Splitting

Designed 70/30 train–test split at pair level; entity leakage is possible

Configuration

Grid search with 5-fold CV (as designed); weights, threshold and seed not visible

Simplified snippets (from the report)

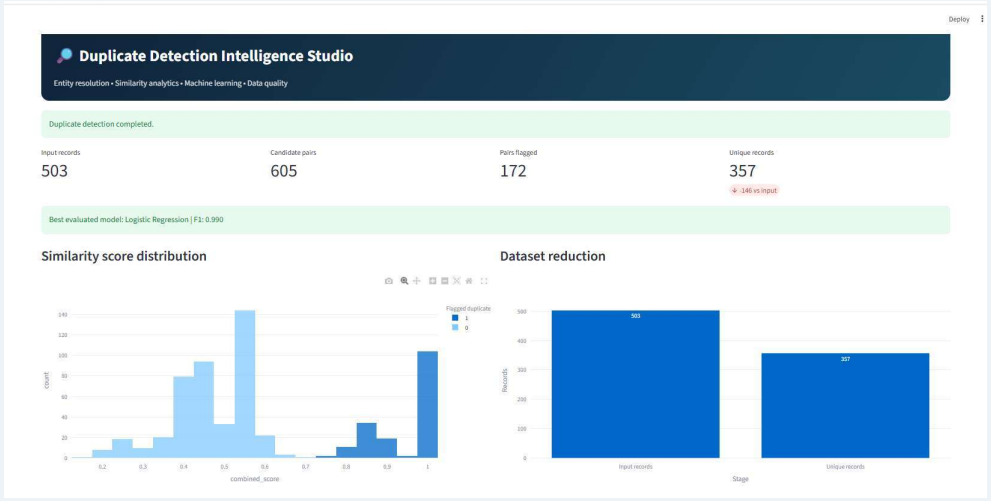
```
# keep digits only
def normalise_phone(v):
    return re.sub(r"\D", "", v or "")

def jaccard(a, b):
    A, B = set(a.split()), set(b.split())
    return len(A & B) / len(A | B) if A | B else 0.0

# union-find: merge predicted pairs
def find(x):
    parent.setdefault(x, x)
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x

for a, b in predicted_duplicate_pairs:
    parent[find(a)] = find(b)
```

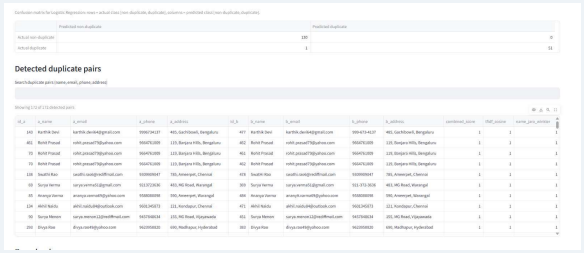
Working Prototype / Output Screens



Dashboard overview
503 records, 605 pairs, 172 flagged, 357 unique



Model evaluation
Three classifiers compared on four metrics



Confusion matrix & pairs
Detected duplicate pairs with scores

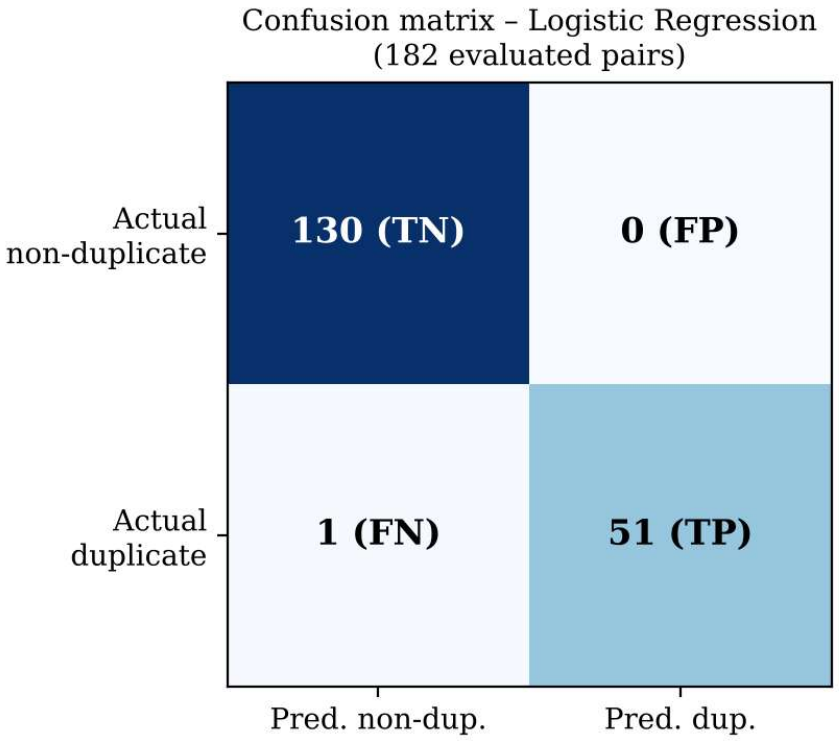
Results & Performance Comparison

Metric	LR	RF	XGBoost
Accuracy	0.995	0.995	0.995
Precision	1.000	1.000	1.000
Recall	0.981	0.981	0.981
F1-score	0.990	0.990	0.990

Best: all three tie – F1 = 0.990

Rounded to three decimals. The dashboard names Logistic Regression best, but the three models tie, so none is declared superior.

Analysis & Key Findings



99.5%

Accuracy of all three models (182 pairs)

Interpretation: 130 TN, 51 TP, 0 FP, 1 FN; precision 1.000, recall 0.981.
Error analysis: One duplicate missed. A false positive merges distinct people; a false negative leaves a duplicate row.
Key drivers: Combined score clusters near 1.0; exact phone and email matches add strong evidence.

Testing & Validation

Test ID	Test Description	Expected Result	Actual Result	Status
TC-02	Normalise “999-673-4137” and “9996734137”	Both become the same string	Same string (documented example)	Documented
TC-03	Run blocking on 503 records	126,253 pairs reduced to 605	605 candidate pairs	Observed
TC-06	Flag duplicates and cluster	172 flagged pairs, 357 representatives	172 pairs, 357 representatives	Observed
TC-08	Upload data without duplicate_of	Threshold applied, no supervised metrics	Designed; not re-run	Designed

Functional Testing

Upload, validation, normalisation, blocking, features, models, clustering, dashboard and downloads.

Validation Method

Designed: 70/30 split and 5-fold CV grid search; displayed run reports 182 pairs.

Performance Testing

Accuracy, precision, recall, F1 and confusion matrix. No formal runtime benchmark was run.

Challenges & Limitations

Challenges Faced

- Messy text → normalisation + five similarity types
- 126,253 pairs → blocking + neighbouring names (605)
- Empty fields → empty strings, row flagged, no match
- Pairs ≠ rows (172 vs 146) → union-find, both reported

Limitations

- Generated data; models may learn its limited transformations
- Weights, threshold, seed and unrounded metrics not visible
- Blocking recall not measured; pair split may leak entities
- One false link can merge entities; local prototype only

Conclusion

1. Built an end-to-end pipeline on 503 generated customer records: normalisation, blocking, 11 similarity features, three classifiers, union-find clustering and a dashboard.

2. 126,253 → 605 candidate pairs; 172 flagged; 357 records kept (146 removed). All models: accuracy 0.995, precision 1.000, recall 0.981, F1 0.990.

3. All objectives O1–O7 are delivered as modules (O5 tuning as designed). Value: a traceable path from raw CSV to a cleaned dataset.

Final Outcome

29.0%

Record reduction (503 → 357 records)
Objectives achieved: 7 of 7 (O5 as designed)

Future Scope & Applications

Improve Performance
Learn feature weights, calibrate thresholds, try transformer matchers

Expand Data / Features
Real labelled data, entity-level splits, blocking-recall measurement

Review Workflow & Interface
Approve / reject / defer queue with active learning

Scale & Secure
Batch, database-backed matching; authentication, audit logs, reversible merges

Applications

- CRM and billing databases
- Customer and service-desk data cleaning
- Teaching entity resolution and record linkage
- Adopters: data analysts, engineers, students, business teams

Team Contribution

Reg. No.	Name	Contribution / Modules Handled
2420030540	Allipuram Akhil	Design, implementation, integration, testing, debugging and documentation
2420030440	D. Surya	Design, implementation, integration, testing, debugging and documentation
2420030546	P. Ruthvik Reddy	Design, implementation, integration, testing, debugging and documentation
GUIDE	Ms. Bonagani Prathusha	Project guide (Assistant Professor, CSE): guidance and review

References

[1] I. P. Fellegi and A. B. Sunter, “A theory for record linkage,” J. Amer. Statist. Assoc., vol. 64, no. 328, 1969.

[2] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” Soviet Phys. Doklady, vol. 10, 1966.

[3] M. A. Jaro, “Advances in record-linkage methodology… Tampa, Florida,” J. Amer. Statist. Assoc., vol. 84, no. 406, 1989.

[4] W. E. Winkler, “String comparator metrics and enhanced decision rules in the Fellegi-Sunter model,” ASA Proc., 1990.

[5] G. Salton and M. J. McGill, Introduction to Modern Information Retrieval. New York: McGraw-Hill, 1983.

[6] P. Christen, Data Matching. Berlin: Springer, 2012.

[7] P. Christen, “A survey of indexing techniques for scalable record linkage and deduplication,” IEEE TKDE, vol. 24, no. 9, 2012.

[8] G. Papadakis et al., “Blocking and filtering techniques for entity resolution: A survey,” ACM Comput. Surv., vol. 53, no. 2, 2020.

[9] M. Bilenko and R. J. Mooney, “Adaptive duplicate detection using learnable string similarity measures,” in Proc. ACM SIGKDD, 2003.

[10] L. Breiman, “Random forests,” Mach. Learn., vol. 45, 2001.

[11] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. ACM SIGKDD, 2016.

[12] S. Mudgal et al., “Deep learning for entity matching: A design space exploration,” in Proc. ACM SIGMOD, 2018.

[13] Y. Li et al., “Deep entity matching with pre-trained language models,” arXiv:2004.00584, 2020.

[14] W. W. Cohen, P. Ravikumar and S. E. Fienberg, “A comparison of string distance metrics for name-matching tasks,” in Proc. IJCAI, 2003.

[15] A. Arasu, V. Ganti and R. Kaushik, “Efficient exact set-similarity joins,” in Proc. VLDB, 2006.

Project Artifacts

Dashboard: Duplicate Detection Intelligence Studio (Streamlit + Flask)

Data: generated customer dataset, 503 rows

Outputs: candidate_pair_scores.csv, duplicate_records_report.csv, cleaned_unique_dataset.csv, model_performance_metrics.json

THANK YOU

Questions & Discussion

Team: 2420030540 | 2420030440 | 2420030546 @klh.edu.in

Guide: Ms. Bonagani Prathusha, Assistant Professor, Dept. of CSE